Part 3: Slow password hashing

Run python3 benchmark_kdfs.py and record the median times. The script warms up each implementation and compares SHA-256, PBKDF2, bcrypt, scrypt, and Argon2id with conservative classroom parameters. Values are not universal: production systems must tune memory and work factors for their own hardware and threat model. Argon2id is generally the preferred modern choice; migration constraints may make the other password KDFs relevant.

Try local registration and verification with python3 register_login_demo.py register student01. The JSON database stores metadata, salt, parameters, and a verifier-not plaintext.

Benchmark example

```
cd /workspace/labs/lab01/part3_password_kdfs
python3 benchmark_kdfs.py --trials 3
```

Output has this shape; your timings will differ:

```
SHA-256 (unsafe)             ... ms
PBKDF2-100k                  ... ms
bcrypt-cost-10               ... ms
scrypt-N=2^14                ... ms
Argon2id-32MiB               ... ms
Results are machine-specific; tune parameters for each deployment.
```

The script performs one warm-up followed by the requested number of measured trials and reports their median. Repeat with more trials and compare the ordering, rather than treating one timing as a universal value:

```
python3 benchmark_kdfs.py --trials 5
```

Registration and login example

Use a fictional password that you can enter again, but do not put it on the command line:

```
python3 register_login_demo.py register student01
Password:
registered
```

```
python3 register_login_demo.py verify student01
Password:
verified
```

Entering a different password during verification should print:

```
authentication failed
```

Inspect the generated local database:

```
python3 -m json.tool auth_db.json
```

Confirm that it contains algorithm metadata, iterations, salt, and verifier,
but no plaintext password. auth_db.json is ignored by Git.

Checkpoint questions

1. How does raising a work factor affect both legitimate verification and an offline attacker?
2. Which demonstrated schemes are designed to consume significant memory?
3. Why should benchmark parameters be tuned on deployment hardware?
4. Why is the SHA-256 timing useful as a baseline but unsafe for storage?